

Iteration 4 Final Deck

JavaFX migration + 9 GoF Design
Patterns

교과서 구조와 실제 코드가 1:1로 보이도록 재구성한 수정판

01 · 기능보다 DP 코드 구조 강화

각 DP마다 역할, method, attribute, 시연 포인트 분리

02 · 깨짐 방지형 레이아웃

큰 표를 쪼개고 코드 블록은 한 장에 하나만 배치

03 · Google Doc + 실제 코드 대조

틀리기 쉬운 Factory/State/UI migration 표현 정정

깨진 이유를 먼저 제거했다

표와 코드가 한 페이지에 과밀하게 들어간 것이 가장 큰 문제였다.

- 1 큰 표는 4~6행 단위로 분리하고, 본문 글자 크기를 14pt 이상으로 유지
- 2 DP 설명은 한 장에 하나의 주장만 배치: 역할 매핑, 코드, 흐름, 시연을 분리
- 3 틀릴 확률이 높은 문구는 실제 Java 코드 기준으로 정정
- 4 발표자가 VS Code에서 바로 찾을 수 있도록 Ctrl+F 키워드까지 남김

발표 포인트

이번 deck은 페이지 수를 늘려 가독성을 확보한 수정판이다.

Google Doc은 가져오되, 코드는 다시 대조했다

문서 내용과 실제 구현이 어긋나는 부분을 정정했다.

Google Doc

Form #1~#3, Section 4~16을 deck의 주요 흐름으로 사용

Actual Code

State, Factory Method, JavaFX migration 표현은 실제 클래스 기준으로 보정

PDF/PPTX

깨진 표와 과밀 코드 페이지를 재설계

App Pattern Guide

앱 안에서 보는 DP 설명도 같은 논리로 맞춤

점수 비중은 DP 코드 강화에 둔다

기능 목록은 짧게, 패턴별 논리 구조를 길게 가져간다.

- 1 기능/변경사항: 무엇을 만들었는지 20%
- 2 UI migration: 왜 JavaFX로 바꿨는지 20%
- 3 DP별 코드 구조: 실제 method와 attribute 흐름 60%
- 4 시연: 화면, 터미널 로그, VS Code 코드가 같은 설명을 하게 구성

각 DP는 5장 고정 구조로 읽는다

교과서 예시와 같은 방식으로 매번 반복된다.

1. 문제

왜 이 패턴이 필요했는가

2. 역할 매핑

GoF role ↔ 우리 클래스

3. 코드

핵심 method와 attribute

4. 호출 흐름

버튼 → control → domain

5. 시연

화면, 로그, Ctrl+F 위치

누적 기능 Inventory — 검색/예약

기능 자체보다 어떤 DP를 노출하는지가 중요하다.

영역	기능	Iteration / DP
Search	직항/왕복/환승/다구간 검색	iter1, iter3, iter4
Search	SEL/TYO/NYC/LON city-code 확장	* iter4 · Composite
Booking	승객정보, 좌석 선택, 결제, 확정	iter1~4
Seat	레그룸/라운지 add-on 요금 누적	* iter4 · Decorator
Payment	카드/KakaoPay/ApplePay/계좌/마일리지	* iter4 · Factory + Adapter

누적 기능 Inventory — 조회/환불/e-Ticket

iter2~4 기능은 데모에서 State, Strategy, Observer와 연결된다.

영역	기능	Iteration / DP
Lookup	회원 예약 조회 / Guest PNR 조회	iter2
Cancel	Confirmed/Ticketed에서 취소 요청	iter2 · State
Refund	Full/Partial/No 환불 정책 선택	iter2 · Strategy
Ticket	plain/html/boarding pass 포맷 렌더	* iter4 · Template Method
Bus	발권 이벤트 후 우등고속 버스 연계	iter3 · Observer

Extension Inventory — 완료된 확장

완료/부분/후속을 구분해 말한다.

Base feature	Extension	Status
Payment	결제수단 routing 확장	Done · Factory
Mileage	외부 Skypass API를 내부 interface로 변환	Done · Adapter
Seat	옵션 조합을 wrapper chain으로 누적	Done · Decorator
e-Ticket	동일 ticket data를 다중 포맷으로 출력	Done · Template
Settings	전역 설정 live re-render	Done · Singleton

Extension Inventory — 한계와 후속

숨기지 않고 범위를 명확히 한다.

항목	현재 상태	발표 표현
Admin auth	backend stub 수준	post-course
e-Ticket PDF	renderer abstraction만 있음	후속 확장 가능
Mileage accrual	차감 중심 구현	post-course
Real Skypass HTTP	Adapter seam 있음	교체 지점 준비
Seat on e-ticket	domain link 제한	known limit

Refactoring / DP Inventory

각 iteration은 refactoring이 DP로 발전한 흐름이다.

Iter	패턴/리팩토링	핵심 클래스
1	DP#1 State	Reservation, ReservationState, 8 concrete states
2	DP#2 Strategy	RefundHandler, RefundPolicy family
3	DP#3 Observer	EventPublisher, EventListener, DomainEvent
4	Refactoring#3	Swing → JavaFX, FXML/CSS/Controller
4	DP#4~#9	Composite, Singleton, Factory, Template, Adapter, Decorator

iter4의 핵심은 UI 노출 + 6개 DP 추가

이전 backend capability를 실제 데모 가능한 화면으로 끌어올렸다.

UI migration

Swing 19파일 중심 구조를 JavaFX FXML/CSS/Controller로 재구성

All 9 patterns

State/Strategy/Observer + iter4 신규 6개가 모두 화면에서 행사됨

Build

Maven wrapper와 javafx-maven-plugin 기반 실행

Demo

결제, 좌석, 마일리지, e-Ticket, 설정, 환승 검색까지 연결

문서의 강한 표현은 발표에서 부드럽게 정정한다

틀렸다고 말하기보다 실제 코드 기준으로 정확히 표현한다.

- 1 State: interface와 concrete state 사이에 AbstractReservationState가 있다
- 2 Factory Method: PaymentProcessorFactory는 선택 helper, Creator는 PaymentMethodProcessor / ItineraryFactory
- 3 UI migration: Boundary 중심 전환이 맞지만 Control glue method가 일부 추가됨
- 4 Domain: 핵심 DP 구조는 유지됐다고 말하는 것이 가장 안전함

발표 포인트

이 페이지는 Q&A 방어용으로 중요하다.

Swing → JavaFX: Boundary를 갈아끼우는 리팩토링

설계 패턴은 Control/Domain에 두고 화면은 FXML/CSS/Controller로 분리했다.

Before

Swing layout code가 화면 구조와 스타일을 함께 가짐

After

FXML = 구조, CSS = 스타일, Controller = 이벤트 처리

Why safe

패턴 객체는 Control/Domain에 있어 UI toolkit 교체 영향이 작음

Demo value

Pattern Guide, Settings, payment/seat/ticket 화면에서 DP가 직접 보임

시연 trigger map 1

기능을 누르면 어떤 패턴이 보이는지 연결한다.

화면	행동	패턴
Search	SEL/TYO/NYC/LON 도시 코드 검색	Composite
Search	환승 포함 검색	Factory Method
Seat	레그룸/라운지 체크	Decorator
Payment	결제수단 선택	Factory Method
Payment	마일리지 잔액/차감	Adapter

시연 trigger map 2

기존 iteration 패턴도 최종 앱 안에서 같이 보여준다.

화면	행동	패턴
Header	상태 badge 변화	State
Lookup/Cancel	환불 정책과 금액 preview	Strategy
Confirmation	e-Ticket format toggle	Template Method
Confirmation	버스 도시 선택 후 발권	Observer
Settings	글꼴/통화/테마 변경	Singleton

전체 구조: Boundary는 JavaFX, 패턴은 Control/Domain

발표에서는 큰 class diagram보다 이 위치 관계를 먼저 잡는다.

Boundary

app.fx, FXML, CSS, screen controllers
사용자 입력과 화면 전환 담당

Control

BookingController, RefundHandler,
payment/itinerary/render services
use case orchestration

Domain

Reservation, State, Payment, Flight, SeatView,
AirportLocation
핵심 상태와 규칙

역할분담은 기능보다 책임 경계로 말한다

보고서/발표에서 빠지기 쉬운 항목이다.

- 1 김정욱: 예약/환불 domain flow, QA issue 정리, 최종 통합 검토
- 2 이재호: JavaFX UI migration, Pattern Guide, 발표 deck/PDF 정리
- 3 김경동: 항공편/여정/버스 연계, demo data와 시연 흐름 점검
- 4 공통: Google Doc 요구사항 대조, 코드-발표 불일치 수정

발표 포인트

실제 팀 내부 역할과 다르면 이 페이지의 이름별 문구만 바꾸면 된다.

State: 왜 필요한가

Behavioral pattern

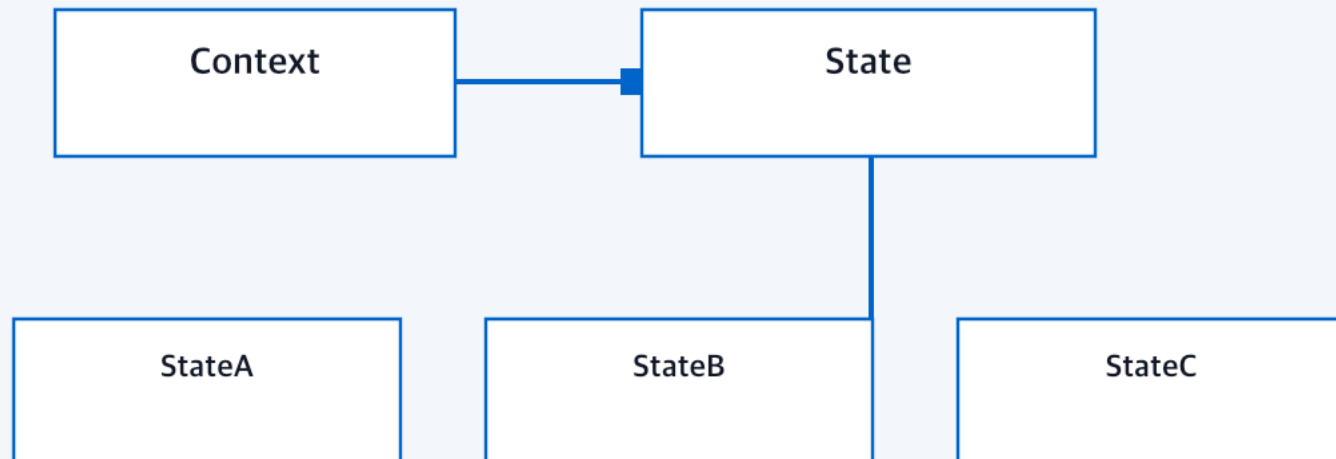
- 1 예약 상태별 허용 동작이 다르다. enum + switch로 처리하면 상태가 늘 때마다 조건문이 퍼진다.
- 2 Context = Reservation / State = ReservationState / Default = AbstractReservationState / ConcreteState = 8개 상태
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Context = Reservation / State = ReservationState / Default = AbstractReservationState / ConcreteState = 8개 상태

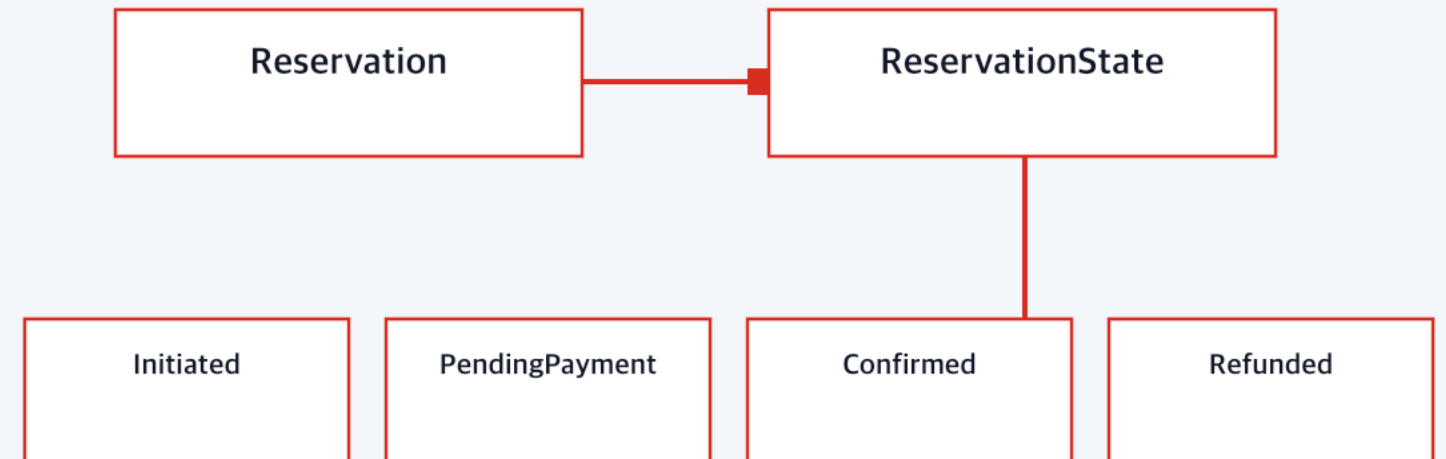
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Context = Reservation / State = ReservationState / Default = AbstractReservationState / ConcreteState = 8개 상태

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
class Reservation {
    private ReservationState currentState;

    public void processPayment() {
        currentState.processPayment(this);
    }

    void setState(ReservationState next) {
        this.currentState = next;
    }
}

abstract class AbstractReservationState
    implements ReservationState {
    public void processPayment(Reservation r) {
        throw new InvalidStateTransitionException(...);
    }
}

class PendingPaymentState extends AbstractReservationState {
    public void processPayment(Reservation r) {
        r.setState(new ConfirmedState());
    }
}
```

핵심 읽는 순서

- 1 currentState가 현재 행동을 결정
- 2 abstract base가 기본 거부를 담당
- 3 concrete state는 허용 전이만 override
- 4 상태 전이는 setState 한 곳으로 집중

Ctrl+F

ReservationState / AbstractReservationState / setState

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 currentState가 현재 행동을 결정
- 2 abstract base가 기본 거부를 담당
- 3 concrete state는 허용 전에만 override

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Header state badge

터미널/로그

상태 전이 로그와
InvalidStateTransitionException

코드에서 찾을 위치

ReservationState, AbstractReservationState,
PendingPaymentState

State는 객체가 자기 전이를 책임지므로 화면은 상태명을 if문으로 판단하지 않습니다.

Strategy: 왜 필요한가

Behavioral pattern

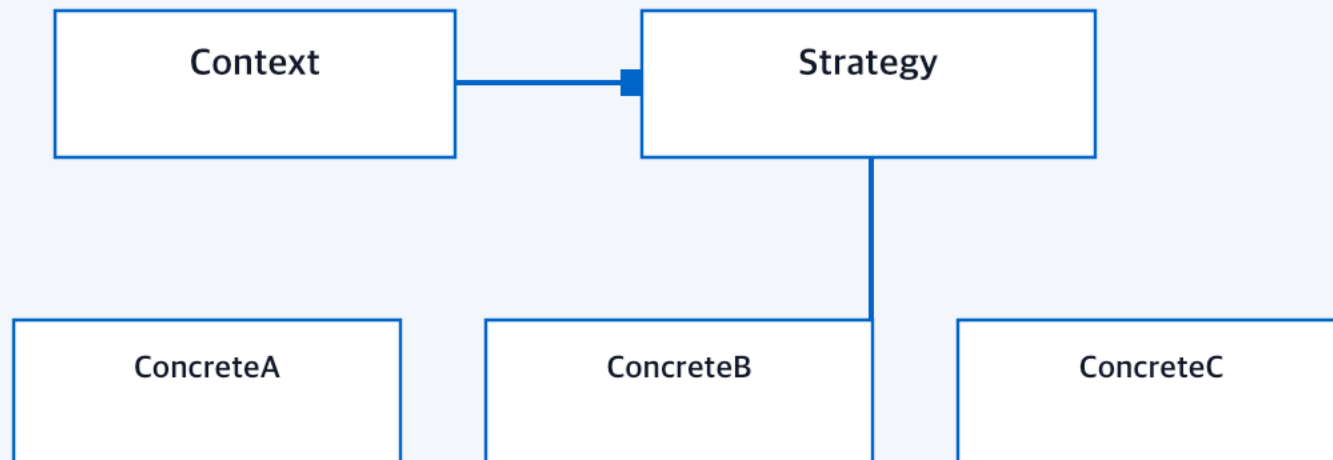
- 1 환불 금액 계산이 fare class별 if/else로 커지면 RefundHandler가 운임 정책 변경에 취약해진다.
- 2 Context = RefundHandler / Strategy = RefundPolicy / ConcreteStrategy = Full, Partial, No
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Context = RefundHandler / Strategy = RefundPolicy / ConcreteStrategy = Full, Partial, No

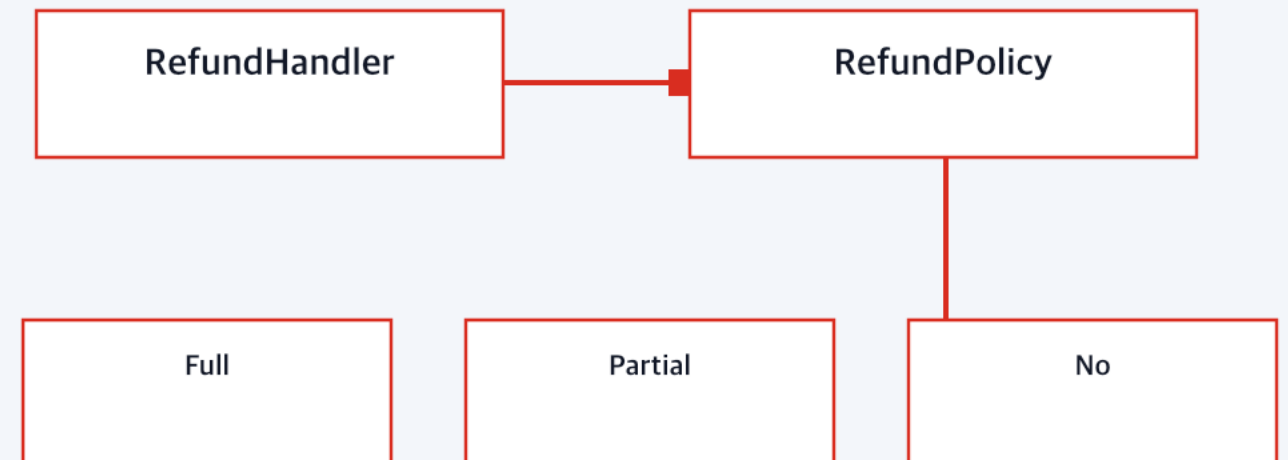
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Context = RefundHandler / Strategy = RefundPolicy / ConcreteStrategy = Full, Partial, No

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
interface RefundPolicy {
    BigDecimal calculateRefundAmount(BigDecimal paid);
    String getRefundType();
}

class FullRefundPolicy implements RefundPolicy {
    BigDecimal calculateRefundAmount(BigDecimal paid) {
        return paid;
    }
}

class PartialRefundPolicy implements RefundPolicy {
    BigDecimal calculateRefundAmount(BigDecimal paid) {
        return paid.subtract(penalty);
    }
}

class RefundHandler {
    RefundRequest evaluateRefund(String pnr, String fareClass) {
        RefundPolicy policy = fareRule.checkRefundPolicy();
        BigDecimal amount = policy.calculateRefundAmount(paid);
        return new RefundRequest(pnr, policy.getRefundType(), amount);
    }
}
```

핵심 읽는 순서

- 1 계산식은 RefundPolicy family로 이동
- 2 RefundHandler는 공통 처리 흐름만 유지
- 3 fare rule이 policy 선택 근거
- 4 새 운임 정책은 concrete strategy 추가로 확장

Ctrl+F

RefundPolicy / calculateRefundAmount / evaluateRefund

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 계산식은 RefundPolicy family로 이동
- 2 RefundHandler는 공통 처리 흐름만 유지
- 3 fare rule이 policy 선택 근거

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Lookup → cancellation/refund

터미널/로그

policy type과 refund amount 로그

코드에서 찾을 위치

RefundHandler / RefundPolicy /
FullRefundPolicy

Strategy는 환불 정책을 갈아끼울 수 있게 해 handler의 if문을 줄입니다.

Observer: 왜 필요한가

Behavioral pattern

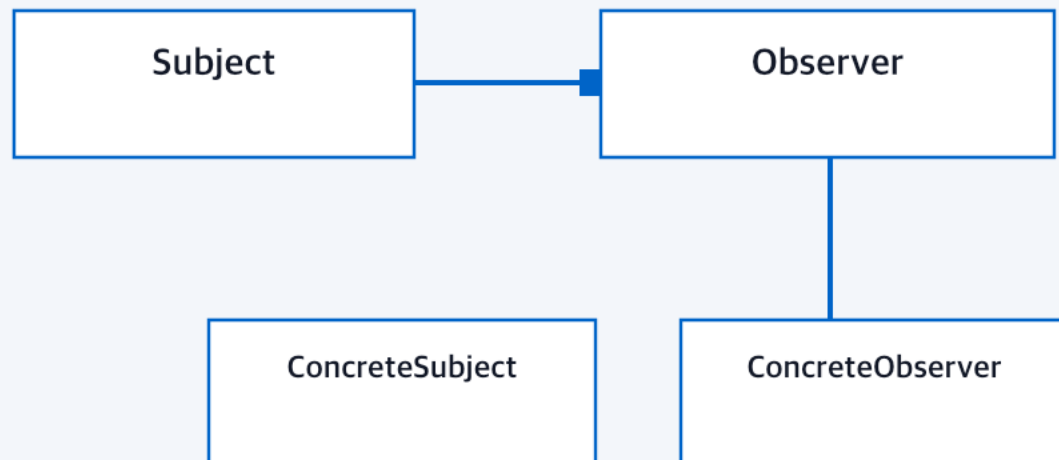
- 1 발권 후 버스티켓, 결제 실패 자동취소 같은 부수효과를 caller가 직접 호출하면 결합도가 높아진다.
- 2 Subject = EventPublisher / Observer = EventListener / Event = DomainEvent
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Subject = EventPublisher / Observer = EventListener / Event = DomainEvent

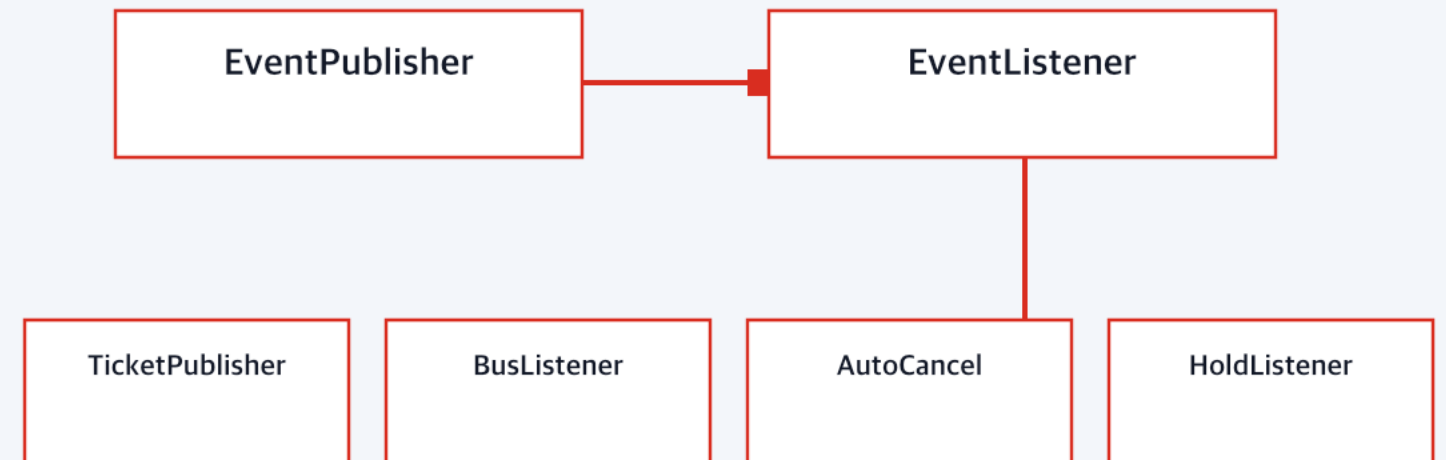
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Subject = EventPublisher / Observer = EventListener / Event = DomainEvent

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
class EventPublisher {
    private final List<EventListener> listeners = new ArrayList<>();

    void register(EventListener listener) {
        listeners.add(listener);
    }

    void publish(DomainEvent event) {
        for (EventListener l : listeners) {
            l.onEvent(event);
        }
    }
}

class TicketPurchasePublisher extends EventPublisher {
    void publishTicketIssued(Reservation r, Ticket t, BusCity city) {
        publish(new TicketIssuedEvent(r, t, city));
    }
}

class BusTicketPurchaseListener implements EventListener {
    public void onEvent(DomainEvent e) {
        if (e instanceof TicketIssuedEvent t) issuePremiumTicket(t);
    }
}
```

핵심 읽는 순서

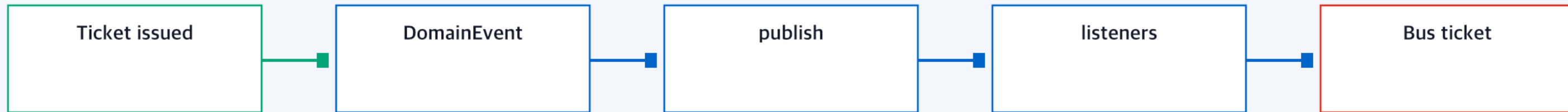
- 1 Subject는 listener의 구체 행동을 모름
- 2 DomainEvent가 통신 단위
- 3 발권/결제실패/홀드만료를 같은 구조로 처리
- 4 버스 연계는 listener 추가로 확장

Ctrl+F

EventPublisher / TicketIssuedEvent /
BusTicketPurchaseListener

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 Subject는 listener의 구체 행동을 모름
- 2 DomainEvent가 통신 단위
- 3 발권/결제실패/홀드만료를 같은 구조로 처리

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Confirmation bus city selector

터미널/로그

[BUS] premium bus ticket ... issued

코드에서 찾을 위치

TicketPurchasePublisher /
BusTicketPurchaseListener

Observer는 발권과 버스 발매를 느슨하게 연결합니다.

Composite: 왜 필요한가

Structural pattern

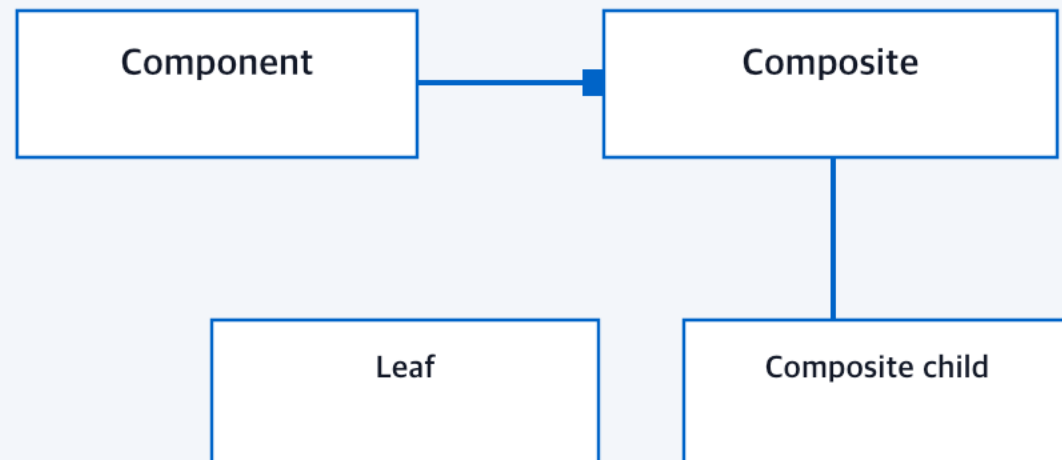
- 1 도시 코드와 공항 코드를 다르게 처리하면 검색 로직이 복잡해진다. TYO는 NRT/HND로 확장되어야 한다.
- 2 Component = AirportLocation / Leaf = Airport / Composite = AirportCity
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Component = AirportLocation / Leaf = Airport / Composite = AirportCity

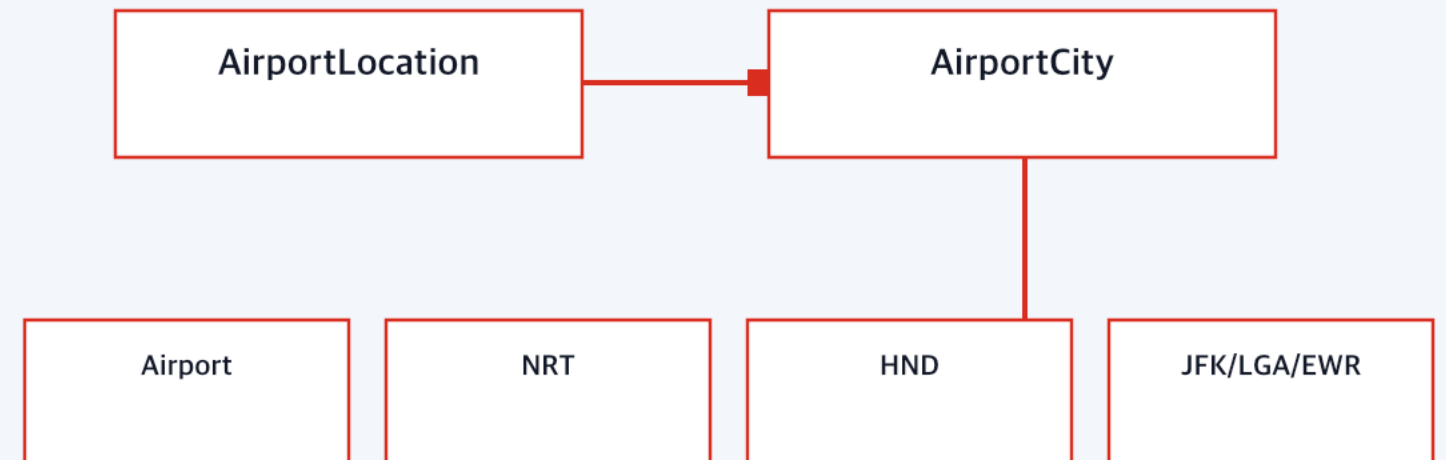
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Component = AirportLocation / Leaf = Airport / Composite = AirportCity

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
interface AirportLocation {
    String getCode();
    List<Airport> getAirports();
    boolean matches(String keyword);
}

class Airport implements AirportLocation {
    public List<Airport> getAirports() {
        return List.of(this);
    }
}

class AirportCity implements AirportLocation {
    private final List<Airport> airports;

    public List<Airport> getAirports() {
        return Collections.unmodifiableList(airports);
    }
}

// Search: resolve("TYO").getAirports() => NRT, HND
```

핵심 읽는 순서

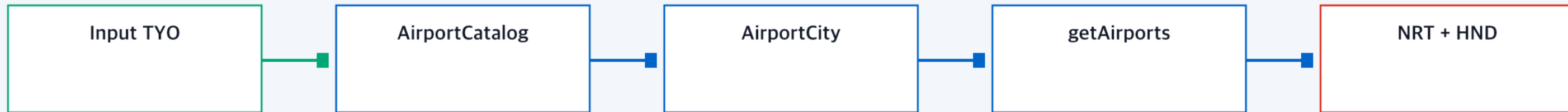
- 1 검색 로직은 AirportLocation만 사용
- 2 Leaf는 자기 자신 1개 반환
- 3 Composite는 하위 공항 목록 반환
- 4 city-code 확장이 if문 없이 가능

Ctrl+F

AirportLocation / AirportCity / getAirports

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 검색 로직은 AirportLocation만 사용
- 2 Leaf는 자기 자신 1개 반환
- 3 Composite는 하위 공항 목록 반환

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Search origin/destination

터미널/로그

검색 결과에 여러 공항 route 표시

코드에서 찾을 위치

AirportLocation / AirportCity / getAirports

Composite는 도시와 공항을 같은 타입으로 다뤄 검색 흐름을 단순화합니다.

Singleton: 왜 필요한가

Creational pattern

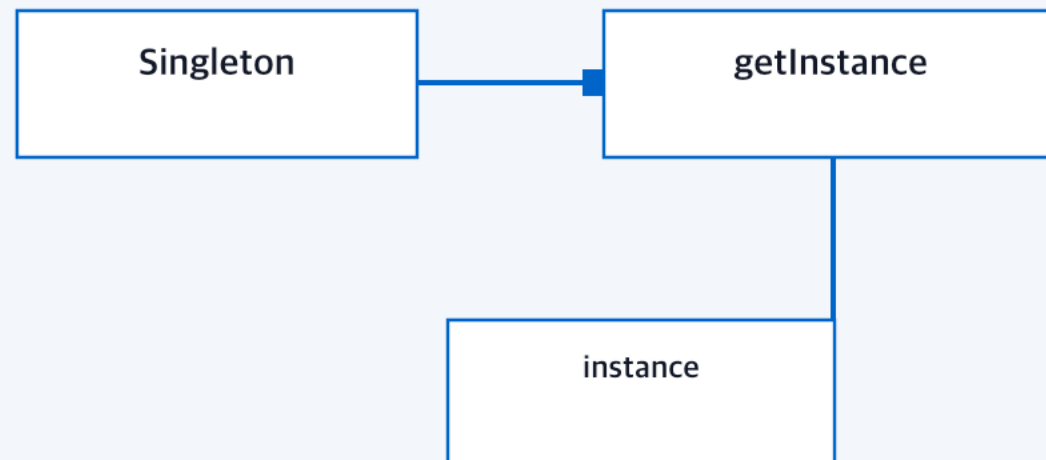
- 1 글꼴, 통화, 테마 설정을 화면마다 따로 가지면 값이 어긋나고 재렌더 흐름이 복잡해진다.
- 2 Singleton = AppConfig / shared instance + listeners
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Singleton = AppConfig / shared instance + listeners

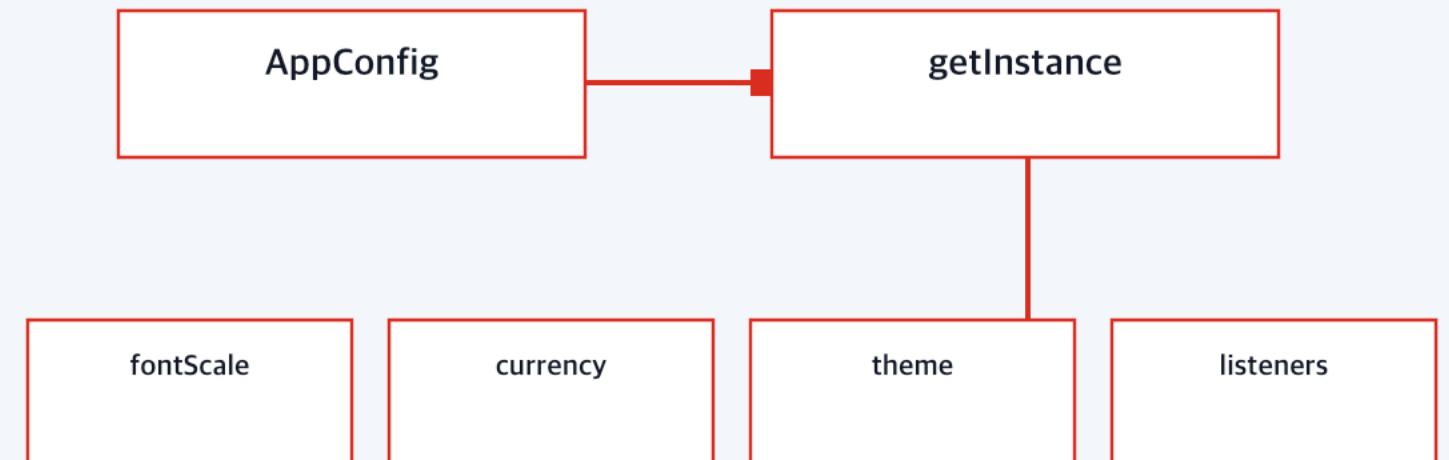
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Singleton = AppConfig / shared instance + listeners

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
public final class AppConfig {
    private static volatile AppConfig instance;
    private final List<Consumer<AppConfig>> listeners;

    private AppConfig() {}

    public static AppConfig getInstance() {
        if (instance == null) {
            synchronized (AppConfig.class) {
                if (instance == null) instance = new AppConfig();
            }
        }
        return instance;
    }

    public void setFontScale(double scale) {
        this.fontScale = scale;
        notifyListeners();
    }
}
```

핵심 읽는 순서

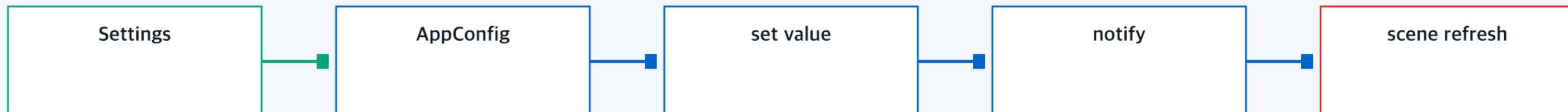
- 1 private constructor로 외부 생성을 차단
- 2 volatile + double checked locking
- 3 설정 변경 후 listener 알림
- 4 모든 JavaFX 화면이 같은 config를 참조

Ctrl+F

AppConfig / getInstance / notifyListeners

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 private constructor로 외부 생성을 차단
- 2 volatile + double checked locking
- 3 설정 변경 후 listener 알림

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Settings screen

터미널/로그

설정 변경 로그 또는 즉시 UI 변화

코드에서 찾을 위치

AppConfig / getInstance / notifyListeners

Singleton은 전역 설정의 단일 진실 공급원입니다.

Factory Method: 왜 필요한가

Creational pattern

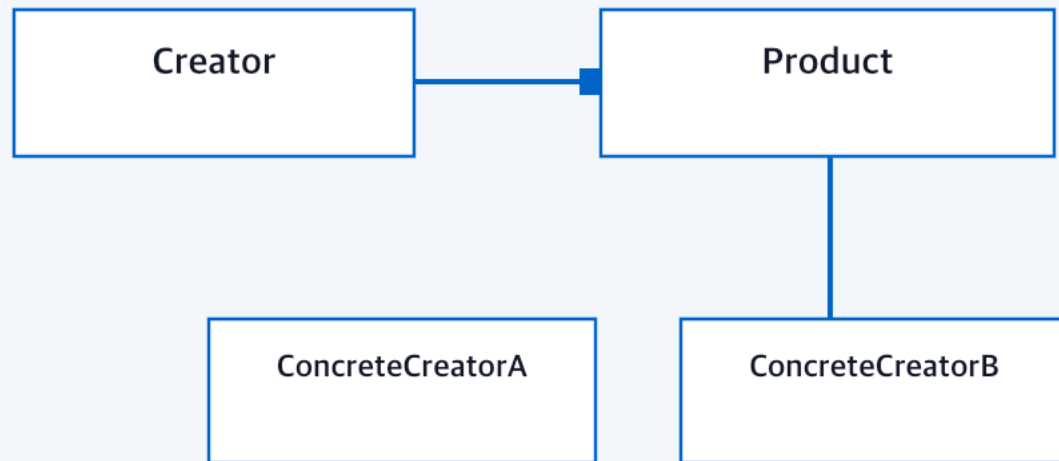
- 1 결제수단과 여정 타입이 늘 때 caller가 concrete class를 직접 new하면 변경 지점이 퍼진다.
- 2 Creator = PaymentMethodProcessor, ItineraryFactory / Factory Method = createPayment, createItinerary
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Creator = PaymentMethodProcessor, ItineraryFactory / Factory Method = createPayment, createItinerary

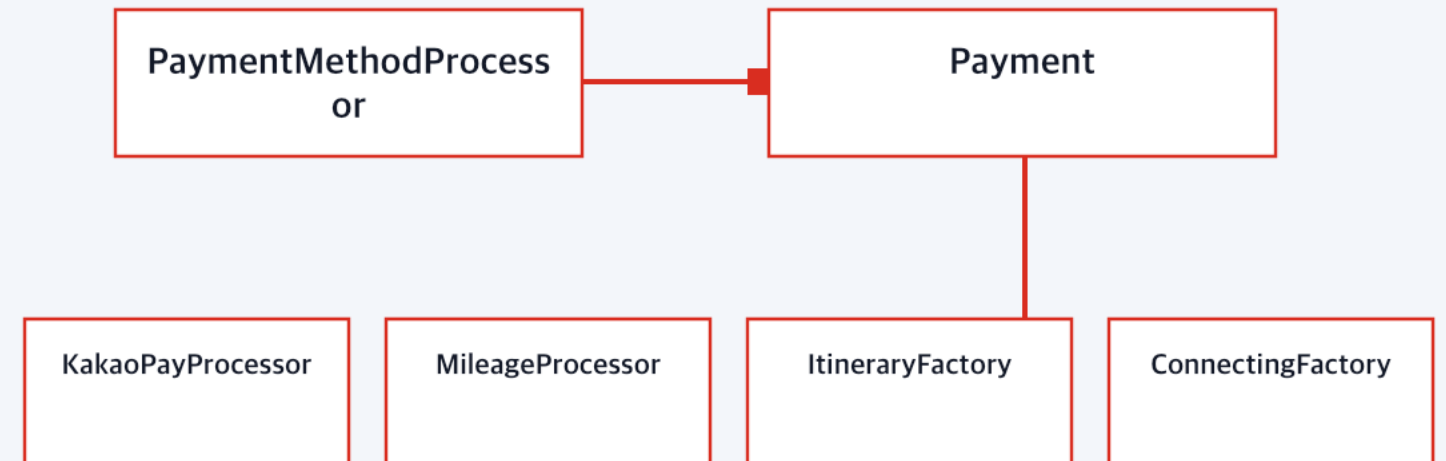
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Creator = PaymentMethodProcessor, ItineraryFactory / Factory Method = createPayment, createItinerary

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
abstract class PaymentMethodProcessor {
    public final Payment processCharge(long amount, String pnr) {
        Payment payment = createPayment(amount);
        if (authorize(payment)) payment.pay();
        else {
            payment.fail();
            publish(new PaymentFailedEvent(payment, pnr, declineReason()));
        }
        return payment;
    }

    protected abstract Payment createPayment(long amount);
}

class KakaoPayPaymentProcessor extends PaymentMethodProcessor {
    protected Payment createPayment(long amount) {
        return basePayment(amount, PaymentMethod.KAKAO_PAY);
    }
}

// PaymentProcessorFactory.forMethod(...) is a selector helper.
```

핵심 읽는 순서

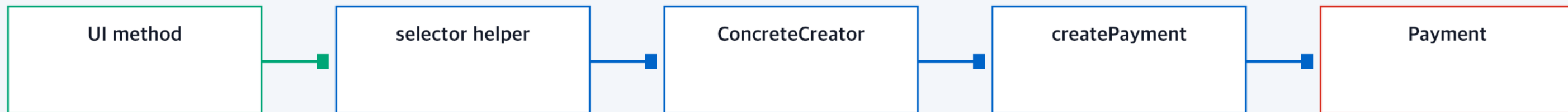
- 1 중요: PaymentProcessorFactory는 선택 helper
- 2 Creator는 abstract processor/factory
- 3 공통 흐름 안에서 factory method 호출
- 4 새 결제수단은 concrete creator 추가

Ctrl+F

PaymentMethodProcessor / createPayment / ItineraryFactory

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 중요: PaymentProcessorFactory는 선택 helper
- 2 Creator는 abstract processor/factory
- 3 공통 흐름 안에서 factory method 호출

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Payment method combo / connecting search

터미널/로그

processor method와 payment status 로그

코드에서 찾을 위치

PaymentMethodProcessor / ItineraryFactory /
createPayment

Factory Method는 생성 책임을 subclass로 밀어 concrete class 의존을 줄입니다.

Template Method: 왜 필요한가

Behavioral pattern

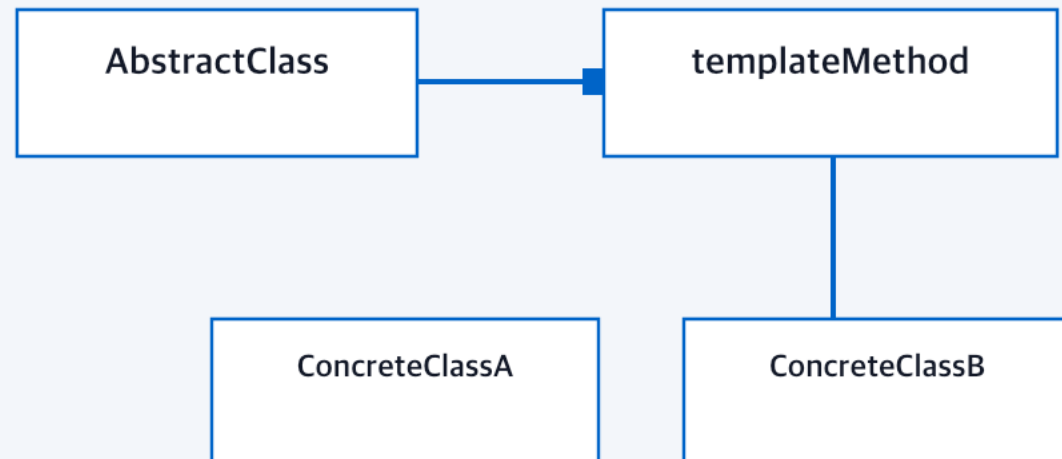
- 1 e-Ticket 포맷은 여러 개지만 header-body-footer 흐름은 항상 같다. 흐름을 중복하면 포맷 추가가 어렵다.
- 2 AbstractClass = TicketRenderer / templateMethod = final render / primitive = header, body, footer
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

AbstractClass = TicketRenderer / templateMethod = final render / primitive = header, body, footer

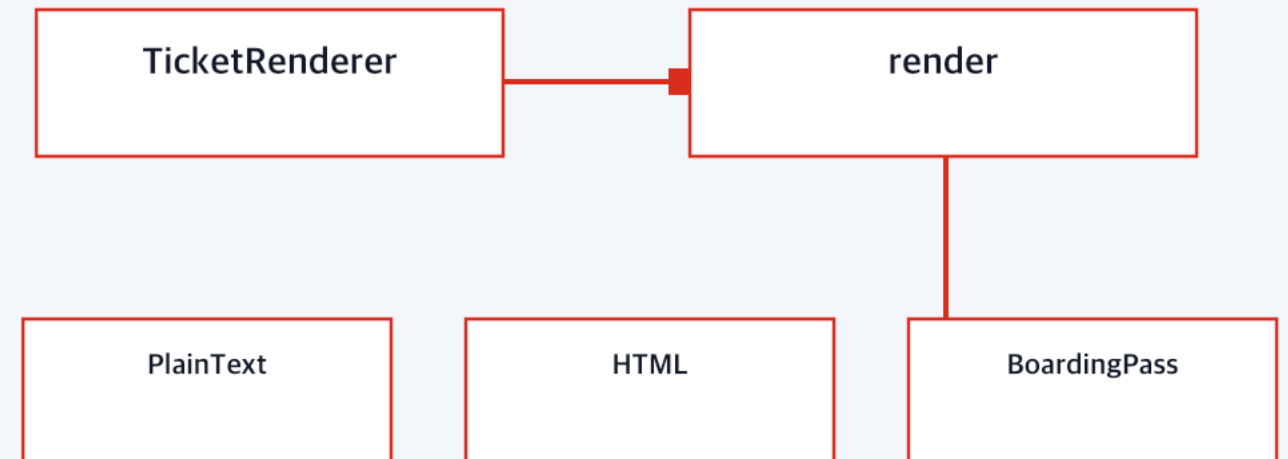
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



AbstractClass = TicketRenderer / templateMethod = final render / primitive = header, body, footer

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
abstract class TicketRenderer {
    public final String render(Reservation r, Ticket t) {
        return header(r, t)
            + separator()
            + body(r, t)
            + footer(r, t);
    }

    protected abstract String header(Reservation r, Ticket t);
    protected abstract String body(Reservation r, Ticket t);
    protected abstract String footer(Reservation r, Ticket t);

    protected String separator() {
        return "\n---\n";
    }
}

class BoardingPassRenderer extends TicketRenderer {
    protected String body(Reservation r, Ticket t) { return boardingPass; }
}
```

핵심 읽는 순서

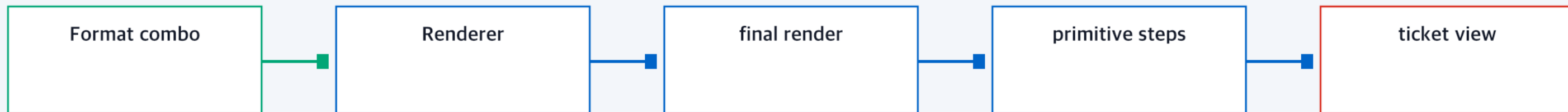
- 1 render는 final로 골격 고정
- 2 format별 세부 출력만 subclass 구현
- 3 separator는 hook처럼 override 가능
- 4 동일 data를 여러 presentation으로 렌더

Ctrl+F

TicketRenderer / final String render / BoardingPassRenderer

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 render는 final로 골격 고정
- 2 format별 세부 출력만 subclass 구현
- 3 separator는 hook처럼 override 가능

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Confirmation ticket format toggle

터미널/로그

plain/html/boarding pass 문자열 변화

코드에서 찾을 위치

TicketRenderer / render /
BoardingPassRenderer

Template Method는 바뀌지 않는 출력 순서를 상위 클래스에 고정합니다.

Adapter: 왜 필요한가

Structural pattern

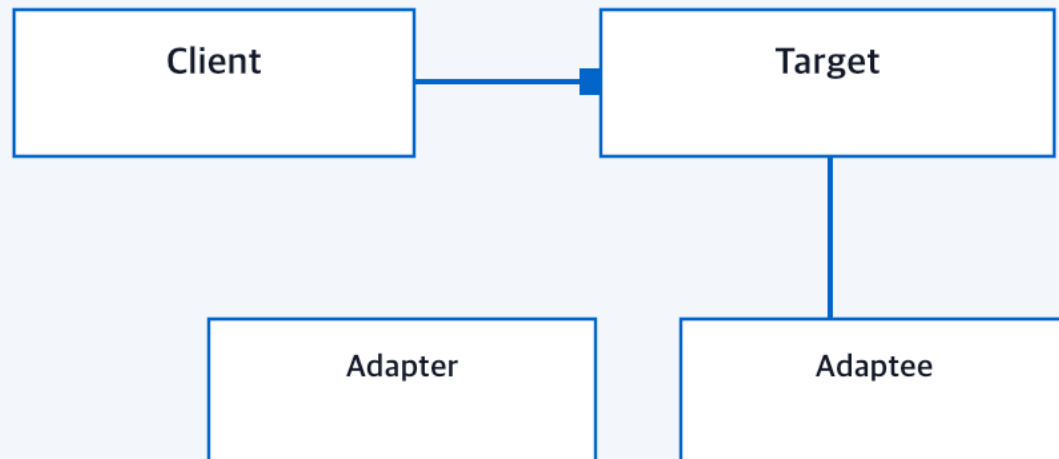
- 1 외부 Skypass API가 Map 기반 응답을 주면 payment screen이 외부 shape에 직접 묶인다.
- 2 Target = SkypassInterface / Adapter = SkypassAdapter / Adaptee = RemoteSkypassApi
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Target = SkypassInterface / Adapter = SkypassAdapter / Adaptee = RemoteSkypassApi

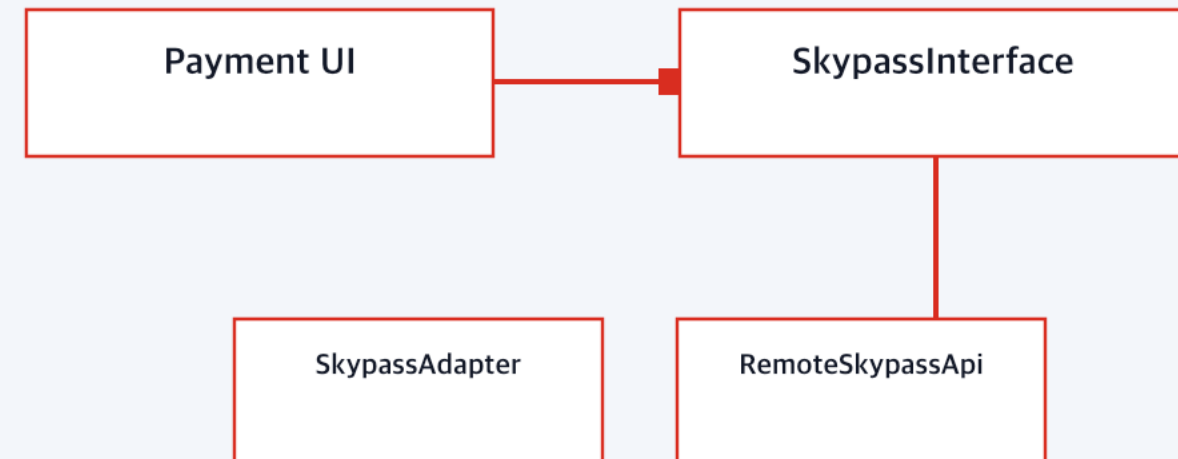
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Target = SkypassInterface / Adapter = SkypassAdapter / Adaptee = RemoteSkypassApi

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
interface SkypassInterface {
    int getMileageBalance(String memberNo);
    Object verifyAndDeduct(String memberNo, int amount);
}

class SkypassAdapter implements SkypassInterface {
    private final RemoteSkypassApi remote;

    public int getMileageBalance(String memberNo) {
        Map<String, Object> res = remote.getMileage(memberNo);
        return ((Number) res.get("balance")).intValue();
    }

    public Object verifyAndDeduct(String memberNo, int amount) {
        Map<String, Object> res = remote.deduct(memberNo, amount);
        return convertToDomainResult(res);
    }
}
```

핵심 읽는 순서

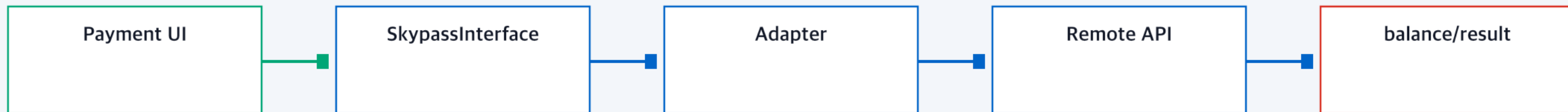
- 1 client는 Target interface만 사용
- 2 Adapter가 external Map을 내부 타입으로 변환
- 3 외부 API 변경 영향이 adapter로 국한
- 4 마일리지 결제 화면에서 바로 보임

Ctrl+F

SkypassInterface / SkypassAdapter / RemoteSkypassApi

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 client는 Target interface만 사용
- 2 Adapter가 external Map을 내부 타입으로 변환
- 3 외부 API 변경 영향이 adapter로 국한

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Mileage balance / mileage payment

터미널/로그

마일리지 잔액, 차감 결과 로그

코드에서 찾을 위치

SkypassInterface / SkypassAdapter /
RemoteSkypassApi

Adapter는 외부 API 모양을 내부 도메인 언어로 바꿉니다.

Decorator: 왜 필요한가

Structural pattern

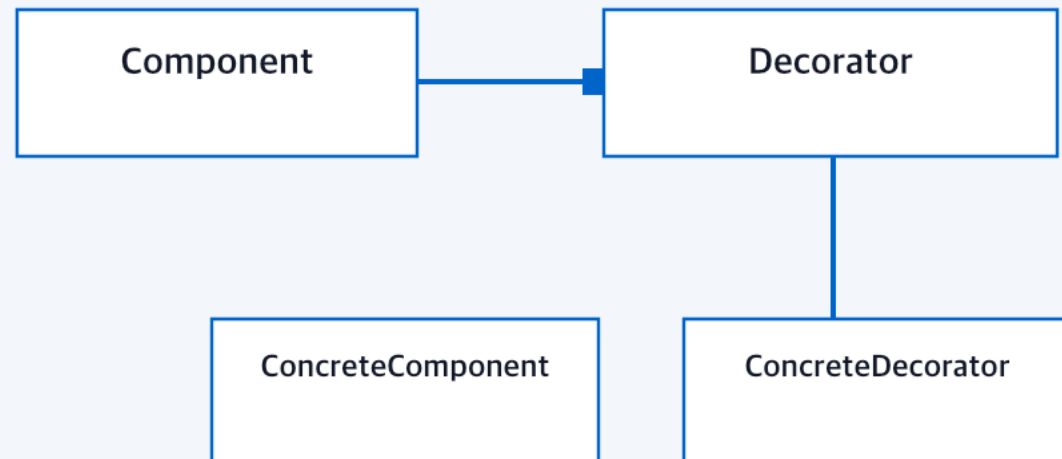
- 1 window/aisle/legroom/lounge 조합마다 class를 만들면 조합 폭발이 일어난다.
- 2 Component = SeatView / ConcreteComponent = SeatViewAdapter / Decorator = AbstractSeatDecorator
- 3 이 장부터 교과서 구조와 실제 Java 코드를 1:1로 연결한다.

Reference vs Team Implementation

Component = SeatView / ConcreteComponent = SeatViewAdapter / Decorator = AbstractSeatDecorator

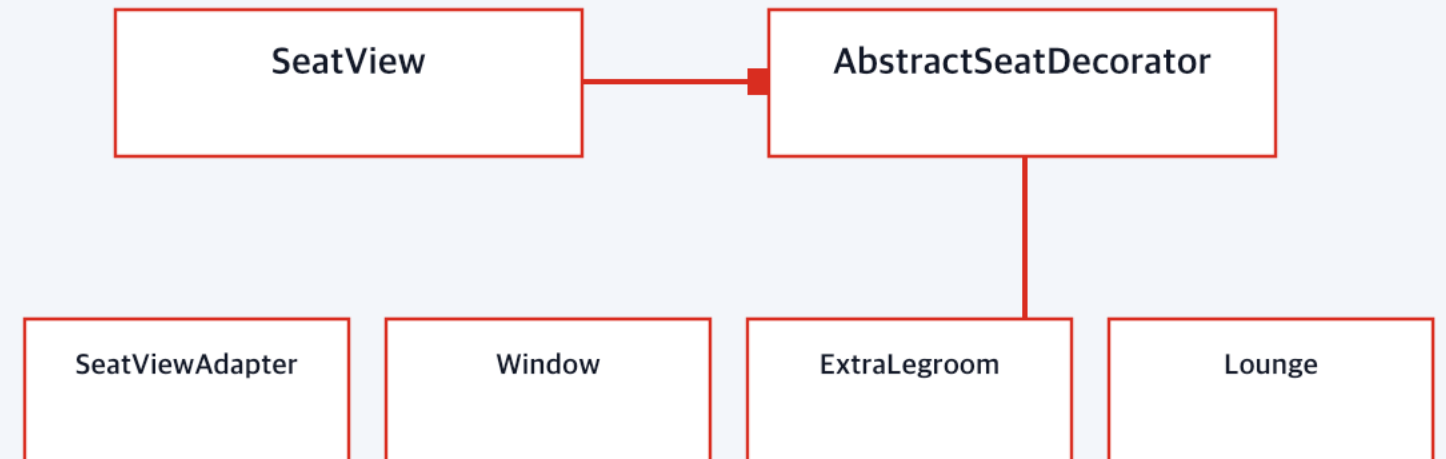
교과서 GoF 구조

교과서 역할 이름을 먼저 읽고



우리 팀 구현

오른쪽에서 우리 클래스명으로 치환한다



Component = SeatView / ConcreteComponent = SeatViewAdapter / Decorator = AbstractSeatDecorator

Code Skeleton

한 장에 핵심 코드 하나만 둔다.

```
SeatView view = new SeatViewAdapter(seat);

if (isWindow(seat)) {
    view = new WindowSeatDecorator(view);
} else if (isAisle(seat)) {
    view = new AisleSeatDecorator(view);
}

if (legroomCheck.isSelected()) {
    view = new ExtraLegroomDecorator(view);
}

if (loungeCheck.isSelected()) {
    view = new LoungeAccessDecorator(view);
}

ctx.setSeatSurcharge(view.getSurcharge().longValue());
seatSummary.setText(view.getDescription());
```

핵심 읽는 순서

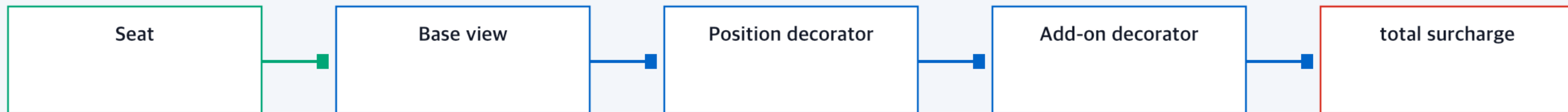
- 1 base SeatView를 wrapper가 감싼다
- 2 description과 surcharge가 누적
- 3 런타임 조합이 가능
- 4 결제 금액으로 add-on이 이어짐

Ctrl+F

SeatView / AbstractSeatDecorator / ExtraLegroomDecorator

Runtime Flow

시연 버튼이 어떤 객체 호출로 이어지는지 설명한다.



- 1 base SeatView를 wrapper가 감싼다
- 2 description과 surcharge가 누적
- 3 런타임 조합이 가능

Demo + Ctrl+F 포인트

앱 화면, 터미널 로그, 코드 위치를 한 번에 연결한다.

앱에서 볼 화면

Seat selection screen

터미널/로그

총액 변화와 선택 옵션 label

코드에서 찾을 위치

SeatView / AbstractSeatDecorator /
SeatViewBuilder

Decorator는 좌석 옵션을 상속이 아니라 조합으로 해결합니다.

Pattern x Iteration

도입된 패턴은 이후 iteration에서도 계속 active로 유지된다.

Pattern	Category	Introduced	iter4 status
State	Behavioral	iter1	active
Strategy	Behavioral	iter2	active
Observer	Behavioral	iter3	active
Composite / Singleton / Factory	Structural / Creational	iter4	* NEW
Template / Adapter / Decorator	Behavioral / Structural	iter4	* NEW

원래 5개 계획에서 9개 패턴으로 확장

추가된 패턴도 앱에서 시연 가능하다.

분류	계획	최종 구현
Core plan	State, Strategy, Observer, Singleton	모두 구현
Optional	Factory Method	Payment + Itinerary에 구현
Beyond plan	-	Composite, Template Method, Adapter, Decorator
Total	5 planned	9 delivered

빨간색으로 짚을 Iteration 4 변경사항

보고서/발표에서 새로 추가된 부분을 시각적으로 구분한다.

New DP

Composite, Singleton, Factory Method, Template Method, Adapter, Decorator

New UI

JavaFX app.fx, FXML, CSS, Pattern Guide

New demo

city-code, payment method, mileage, seat add-on, ticket format, settings

QA fixes

double refund, fare scaling, navigation/session, connecting path

QA: 발견한 문제와 수정 방향

교수님이 물어볼 수 있는 안정성 질문 대응.

Issue	Risk	Fix
Double refund	자동 환불과 review가 중복	immediate cancel과 review path 분리
Fare hardcoding	다구간 금액 부정확	segment count 기반 scaling
Back navigation	세션/상태 유실	AppContext/Navigator 경로 보강
Connecting booking	multi-city로 오인	ConnectingItineraryFactory path 추가
Pattern mismatch	발표와 코드 불일치	Pattern Guide와 deck 표현 정정

의도적 한계는 숨기지 않는다

범위를 명확히 말하면 오히려 설계가 탄탄해 보인다.

Admin auth

manual refund review는 있으나 admin 인증은 stub

Single passenger

현재 flow는 성인 1명/economy 중심

Ticket PDF

Renderer abstraction은 있으나 PDF download는 후속

External API

Skypass는 adapter seam만 실제 교체 가능하게 준비

시연 순서: 사용 흐름 속에서 9개 DP를 모두 지나간다

중간에 코드 Ctrl+F를 섞어 설명하면 논리 흐름이 끊기지 않는다.

- 1 Search: SEL/TYO/NYC/LON 검색으로 Composite 설명
- 2 Seat: 좌석 선택 + add-on으로 Decorator 설명
- 3 Payment: 결제수단 + 마일리지로 Factory Method와 Adapter 설명
- 4 Confirmation: e-Ticket format + bus ticket으로 Template와 Observer 설명
- 5 Lookup/Cancel/Refund: State badge와 Strategy policy 설명
- 6 Settings/Pattern Guide: Singleton과 전체 DP map 마무리

틀리기 쉬운 질문은 코드 기준으로 답한다

이 세 문장은 그대로 외워도 좋다.

Factory Method

`PaymentProcessorFactory`는 selector helper이고, Creator는
`PaymentMethodProcessor`와 `ItineraryFactory`입니다.

State

`AbstractReservationState`를 둔 이유는 모든 invalid transition 기본 동작을
중복 없이 모으기 위해서입니다.

UI migration

Boundary 중심 전환이며, Control에는 화면 노출을 위한 얇은 glue만
추가했습니다.

Strategy

`RefundHandler`만이 아니라 `RefundPolicy` concrete family까지 보여줘야
Strategy가 보입니다.

최종 결론: 기능 확장 축을 패턴으로 분리했다

발표의 마지막 문장.

- 1 기능이 늘어날수록 if문으로 버티지 않고, 변하는 부분을 객체 역할로 분리했다.
- 2 JavaFX UI는 사용자가 그 패턴들을 실제로 실행해볼 수 있게 만든 Boundary다.
- 3 따라서 앱 시연, 터미널 로그, VS Code 코드가 같은 구조를 가리킨다.